

Process Abstraction, Kernel State, and Context Switching

Week 3: Processes, threads, lifecycle, task records, and observation

SDB

Autumn 2026

Intermediate/graduate engineering lecture set

Learning outcomes

By the end of this lecture set, you should be able to:

1. distinguish programs, processes, threads, address spaces, and kernel tasks;
2. explain lifecycle transitions using runnable and blocked *threads*;
3. identify process-wide versus thread-specific kernel state;
4. trace `fork()`, `execve()`, `exit`, waiting, reparenting, and job control;
5. describe the direct and microarchitectural costs of a context switch;
6. reason about multicore placement, namespaces, cgroups, and resource accounting; and
7. interpret Linux `/proc` and `ps` output without mistaking it for a literal PCB dump.

Guiding question

What state must the kernel preserve so that an execution can stop now and resume correctly later—possibly on another CPU?

Road map

- 1 The Process Abstraction
- 2 Lifecycle, Creation, and Relationships
- 3 Kernel Records and Scheduling State
- 4 Multicore Placement and Resource Control
- 5 Observing Processes on Linux
- 6 Synthesis

Road map

- 1 The Process Abstraction
- 2 Lifecycle, Creation, and Relationships
- 3 Kernel Records and Scheduling State
- 4 Multicore Placement and Resource Control
- 5 Observing Processes on Linux
- 6 Synthesis

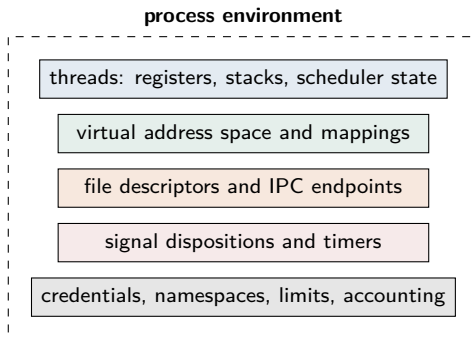
Program, process, and thread are different abstractions

Concept	Meaning	Representative state
Program	passive executable/data format stored or generated	instructions, metadata, interpreter request
Process	protected resource and execution environment	address space, handles, credentials, namespaces
Thread	schedulable instruction stream within a process	registers, stack, scheduling state, TLS
Address space	virtual-memory translation/protection context	mappings, page tables, VM policies
Kernel task	implementation-specific schedulable/managed entity	links to shared and per-thread records

Most important correction

On mainstream general-purpose systems, the scheduler selects runnable *threads/tasks*, not an indivisible process. A multithreaded process can have one thread running while another is blocked.

A process is a bundle of shared resources



- The process abstraction supplies isolation, naming, accounting, and a unit for resource ownership.
- Threads deliberately share much process state, enabling low-cost communication but also shared failure fate.
- The kernel may distribute this state across many objects rather than store one monolithic PCB.

Design lens

Ask for each field: Is it per-thread, process-shared, reference-counted, inherited, copied, reset, or replaced?

Identity is contextual, not one universal PID

Process-related identities can include:

- thread ID, process/thread-group ID, parent ID;
- process-group ID and session ID;
- real/effective/saved user and group IDs;
- namespace-relative PIDs;
- stable kernel handles such as Linux pids;
- security labels and audit identities.

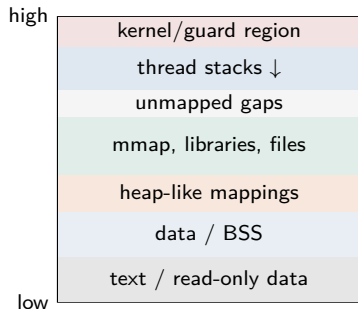
PID reuse

A numeric PID is a temporary name. Between lookup and action, the original process can exit and the number can be reused. Handle-based interfaces reduce this race.

Namespace consequence

The same task can have different PID values when viewed from nested PID namespaces.

Virtual address space: mappings, not a fixed four-segment box



- Virtual memory is sparse; regions are independent mappings with permissions and backing objects.
- Shared libraries, files, anonymous arenas, JIT code, shared memory, and multiple thread stacks complicate the textbook picture.
- ASLR changes placement; exact layout depends on architecture, ABI, loader, runtime, and security policy.
- Heap and stack growth directions are conventions, not defining properties of a process.

Address zero

Low virtual addresses are commonly left unmapped to catch null dereferences; user text does not generally begin at exactly zero.

Threads: shared state and private execution context

Normally shared within a process

virtual address space and code
heap/global memory mappings
open-file table / many handles
current directory and many credentials
signal dispositions
resource limits and namespaces

Normally per thread

general registers, PC, SP, flags
user and kernel stack
scheduling class, priority, affinity
thread-local storage and robust-list state
signal mask and pending thread-directed signals
CPU accounting and fault/debug state

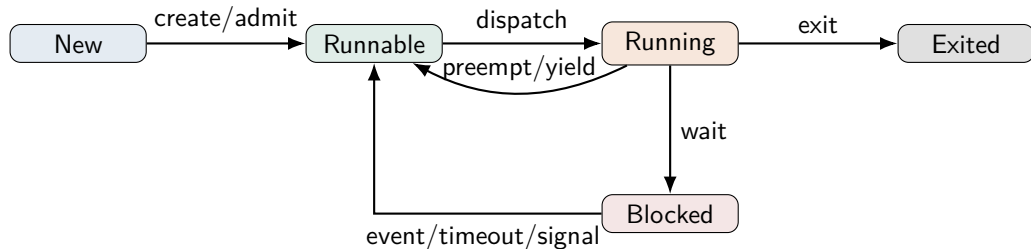
Consequence

Blocking one thread does not block its siblings unless they depend on the same lock, resource, or process-wide condition. One thread's memory corruption can damage every thread in the process.

Road map

- 1 The Process Abstraction
- 2 Lifecycle, Creation, and Relationships**
- 3 Kernel Records and Scheduling State
- 4 Multicore Placement and Resource Control
- 5 Observing Processes on Linux
- 6 Synthesis

Thread lifecycle: portable conceptual model



- **Runnable** combines “ready” and currently executing; only one thread per logical CPU is running at an instant.
- **Blocked** means an event is required before the thread can run; it is not simply waiting its turn for a CPU.
- Stop/tracing, uninterruptible sleep, parking, and zombie states are OS-specific refinements.

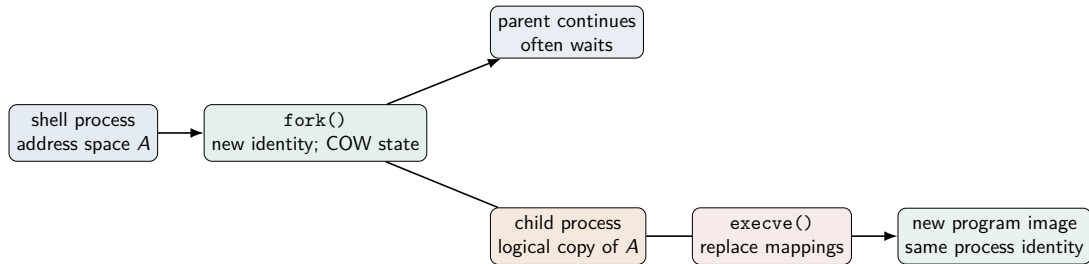
Linux task states do not map one-to-one to the textbook model

Code	Typical ps meaning	Interpretation
R	running or runnable	on a CPU or eligible for scheduling
S	interruptible sleep	waiting; signals can wake/interrupt
D	uninterruptible sleep	commonly waiting in a kernel path; not safely interruptible at that point
T/t	stopped or traced	job-control stop or debugger/tracer stop
Z	zombie	thread-group leader/process exit status awaits reaping
I	idle kernel thread	kernel-specific idle state in some tools

Snapshot caveat

State can change immediately after observation. Tools also aggregate or choose one thread's state; inspect per-thread data when diagnosing a multithreaded process.

Creating a process image: `fork()` then `execve()`

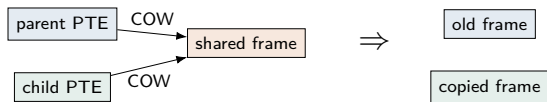


- `fork()` creates a child and returns in both parent and child; memory copy semantics are commonly implemented with copy-on-write.
- `execve()` does not create a new process. It replaces the calling process's program image and resets/preserves attributes according to a detailed contract.

Copy-on-write: semantics versus implementation

Immediately after `fork()`:

- parent/child PTEs reference shared frames;
- writable mappings become logically COW;
- page tables and metadata still require work;
- a write fault copies only the modified page.



Large processes can still pay for page-table duplication, reference accounting, TLB work, and later COW faults.

What `execve()` replaces and what it preserves

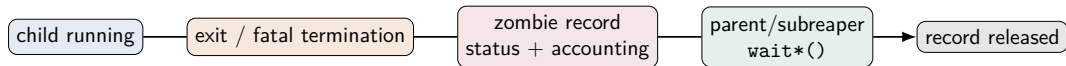
Replaced/reset examples	Preserved examples
user address-space mappings and code thread set becomes one calling thread caught signal dispositions reset to defaults alternate signal stack and many runtime objects program arguments/environment are rebuilt architecture execution entry state	PID and parent relationship many open descriptors unless close-on-exec current directory and root directory resource limits and selected credentials process group, session, controlling terminal accounting/history defined by the OS

Security-critical detail

Descriptor leakage across `execve()` can unintentionally grant the new program access to files, sockets, pipes, or privileged service endpoints. Prefer atomic close-on-exec creation flags.

The exact contract includes exceptions for privilege transitions, tracing, secure execution, and OS-specific attributes; consult the platform specification.

Exit, zombie state, and reaping

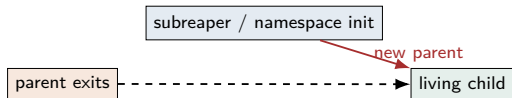


- Most execution resources are released at exit, but a small kernel record remains so an authorized waiter can collect status and usage.
- A zombie does not execute and does not retain its ordinary user address space.
- Persistent zombies indicate a parent/reaper bug and consume process-table/PID-related resources.

Orphans, subreapers, and PID namespaces

If a parent exits before a living child:

- the child continues unless another condition terminates it;
- it is reparented to an appropriate child subreaper or namespace init process;
- the new reaper becomes responsible for collecting its later exit status.

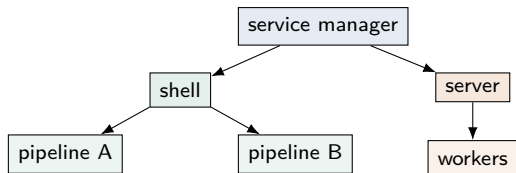


An orphan is not inherently erroneous; daemons and service supervisors intentionally manage parent/child lifecycles.

Correction

The new parent is not necessarily the host's PID 1 or `systemd`. Subreapers and nested PID namespaces alter the adoption path.

Process trees are not scheduling hierarchies



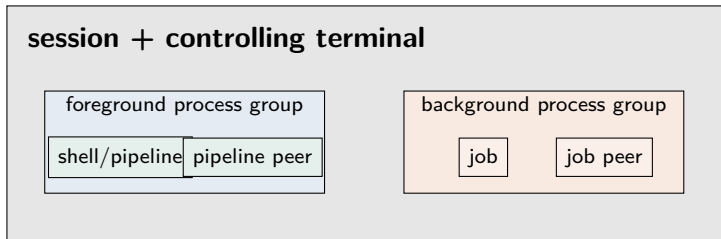
Parent/child relationships support creation, signals, waiting, and supervision. They do *not* necessarily define:

- CPU-share allocation or scheduler groups;
- memory/resource-control hierarchy;
- security trust or ownership;
- terminal foreground/background membership.

Separate structures

cgroup trees, namespace relationships, process groups, sessions, and service-manager dependency graphs serve different purposes.

Process groups, sessions, and terminal job control



- The terminal driver directs generated job-control signals to the foreground process group.
- Background execution means shell/job-control status, not independence from all terminal effects or parent relationships.
- `Ctrl-Z`, `fg`, and `bg` manipulate stopped jobs and process groups, not individual scheduler states alone.

Road map

- 1 The Process Abstraction
- 2 Lifecycle, Creation, and Relationships
- 3 Kernel Records and Scheduling State**
- 4 Multicore Placement and Resource Control
- 5 Observing Processes on Linux
- 6 Synthesis

There is rarely one literal “PCB” structure

PCB is a useful conceptual name for kernel-maintained state needed to manage a process. Production kernels commonly decompose it into:

- per-thread scheduling and saved execution state;
- shared memory-management objects;
- file/handle tables and filesystem context;
- credentials, namespaces, signals, timers;
- parent/child and process-group relationships;
- accounting, tracing, security, and resource-control links.

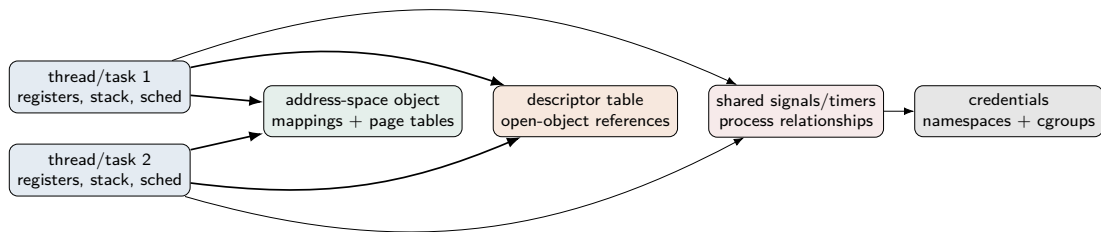
Why decomposition?

Sharing, copy-on-write, independent lifetime, cache locality, locking discipline, and feature evolution all favor referenced objects over one giant record.

Terminology

Use “task/thread control block” for schedulable state and “process-wide state” for shared resources when precision matters.

Conceptual kernel object graph



- Reference counts and locks coordinate shared lifetime and concurrency.
- `fork/clone` semantics determine which objects are copied, shared, or newly allocated.
- Exit can release some objects immediately while retaining identity/accounting for waiting.

What must be saved for a context switch?

Architectural/thread state

- program counter, stack pointer, flags;
- callee/architecture-required registers;
- floating-point/vector state as required;
- debug, TLS, control, and security state;
- kernel stack location and return path.

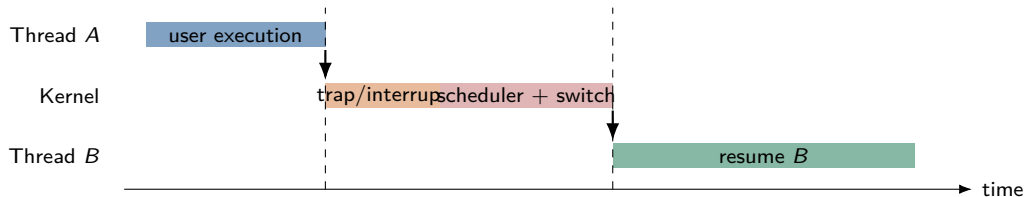
Execution environment

- address-space/page-table context when it changes;
- CPU-local scheduler/current-task pointer;
- accounting, tracing, and lock/preemption state;
- architecture mitigations and lazy/eager state policies.

Important nuance

A switch between threads in one process may preserve the address space. A switch between processes may use ASID/PCID-tagged translations rather than flushing the entire TLB.

Context-switch timeline



1. An interrupt, exception, syscall, block, or explicit yield enters the kernel.
2. The current thread's necessary state is preserved; scheduler policy may select another runnable thread.
3. Low-level code changes task/stack/address-space context and returns into the selected thread.

Context-switch cost is direct plus indirect

Direct latency

- entry/exit and scheduler execution;
- saving/restoring register state;
- run-queue locking/accounting;
- address-space or mitigation work.

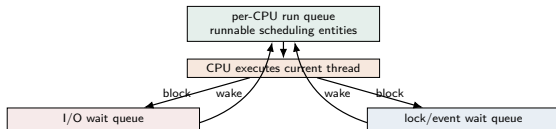
Indirect latency

- cold/private cache lines;
- TLB and page-walk effects;
- branch predictor/prefetch disruption;
- NUMA/cache migration cost;
- contention induced by extra concurrency.

Measurement discipline

There is no universal context-switch time. Specify architecture, kernel, mitigation/configuration, thread relationship, CPU migration, cache state, workload, and metric.

Run queues and wait queues serve different questions



- A run queue answers: *which runnable task should this CPU execute?*
- A wait queue links sleepers to the event or condition that can make them runnable.
- Modern kernels use multiple scheduler-class data structures, per-CPU queues, load balancing, timers, and wakeup placement.
- “Job queue” is a historical batch abstraction, not a universal production-kernel structure.

Intrusive queues and task lifetime

A task record may contain embedded linkage for several structures:

- scheduler/run-queue nodes;
- parent/child and sibling lists;
- wait queues and timers;
- process-group/session membership;
- tracing, cgroup, and namespace indexes.

Why intrusive structures?

They avoid separate allocation and support constant-time linkage, but require strict invariants: one link field cannot simultaneously belong to incompatible lists.

Concurrency challenge

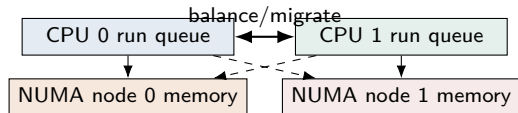
Task lookup, exit, wakeup, migration, and observation race across CPUs. Kernels combine locks, reference counts, RCU-like techniques, and carefully ordered state transitions.

A task can disappear while another subsystem is trying to inspect it; stable references matter.

Road map

- 1 The Process Abstraction
- 2 Lifecycle, Creation, and Relationships
- 3 Kernel Records and Scheduling State
- 4 Multicore Placement and Resource Control**
- 5 Observing Processes on Linux
- 6 Synthesis

Multicore scheduling: placement is part of process performance



- Per-CPU run queues reduce global contention but require load balancing.
- Affinity can preserve cache/NUMA locality; excessive pinning can create imbalance.
- Wakeup placement considers idle CPUs, cache history, topology, energy, and policy.
- Simultaneous multithreading shares execution resources even when logical CPUs are distinct.

Migration cost

Moving a runnable thread is cheap in bookkeeping terms but can be expensive through lost cache locality and remote-memory access.

Resource limits, accounting, and cgroups

Mechanism	Scope	Purpose
per-process limits	inherited soft/hard limits	cap descriptors, memory, CPU time, process creation
credentials/capabilities	task/process security context	authorize operations and reduce privilege
accounting	per-thread/process/group counters	usage reporting, billing, diagnosis, policy feedback
cgroups	hierarchical groups of tasks	CPU share/quota, memory pressure, I/O control, process counts
affinity/cpusets	allowed CPU and memory nodes	placement, isolation, NUMA policy
service-manager policy	supervised unit/container	restart, dependency, lifetime, environment

Hierarchy distinction

A cgroup hierarchy is independent of the parent/child process tree. A service can regroup descendants, and unrelated processes can share one resource-control group.

Namespaces virtualize a process's view

Linux namespaces can isolate views of:

- process IDs and parent relationships;
- mounts/filesystem namespace;
- networking interfaces/routes/ports;
- users and capability interpretation;
- IPC objects, hostname, and time domains.

Container connection

A container is a set of ordinary processes constrained by namespaces, cgroups, credentials, filesystem setup, and security policy. It is not a special process state.

Shared kernel

Namespace isolation changes visibility and naming; it does not create a separate kernel or eliminate shared-kernel attack surfaces.

Signals, cancellation, and process-wide effects

- Signals can be process-directed or thread-directed; delivery selects an eligible thread under platform rules.
- Each thread has a signal mask, while many dispositions are process-shared.
- Stop, continue, termination, and core-dump actions can affect the entire thread group.
- A blocked syscall may return EINTR, be transparently restarted, or complete partially depending on the call and signal configuration.
- Graceful process termination is a protocol: stop accepting work, cancel/wake threads, release resources, and report status.

Asynchrony

State observed before sending a signal may already be stale; signal delivery and handler execution are not instantaneous transaction boundaries.

Road map

- 1 The Process Abstraction
- 2 Lifecycle, Creation, and Relationships
- 3 Kernel Records and Scheduling State
- 4 Multicore Placement and Resource Control
- 5 Observing Processes on Linux
- 6 Synthesis

/proc is a generated interface, not a PCB memory dump

procfs exposes kernel-generated views such as:

- /proc/PID/status: identity, state, memory summaries, capabilities;
- /proc/PID/stat: compact counters/state for tools;
- /proc/PID/maps: virtual-memory mappings;
- /proc/PID/fd: descriptor symlinks;
- /proc/PID/task/TID: per-thread views;
- /proc/PID/ns and cgroup: isolation/resource context.

Three caveats

1. Fields are formatted projections, not raw structures.
2. Separate reads need not form one atomic snapshot.
3. Permission, namespaces, hidepid policy, and process exit can limit or race access.

Thread inspection

For scheduling diagnosis, inspect /proc/PID/task/TID rather than assuming one process-wide state.

A practical observation workflow

```
1 ps -eLo pid,tid,ppid,pgid,sid,psr,stat,ni,comm --sort=pid,tid
2 cat /proc/$$/status
3 cat /proc/$$/sched
4 ls -l /proc/$$/fd
5 sed -n '1,20p' /proc/$$/maps
6 grep -E 'State|Pid|Tgid|Threads|Cpus_allowed_list' /proc/$$/status
```

Ask:

- Which TIDs share a TGID?
- Which CPU last ran each thread?
- Which mappings and descriptors are shared?

Do not conclude:

- R means continuously consuming a CPU;
- a counter explains causality;
- one sample represents steady behavior.

Interpreting common /proc/PID/status fields

Field	Meaning	Caution
Pid/Tgid/PPid	thread/process-group/parent identities	namespace-relative and reusable
State	current task state summary	instantaneous and per task
Threads	number of threads in the thread group	can change concurrently
VmSize	virtual address-space size summary	not physical memory consumption
VmRSS	resident-set estimate/summary	shared pages and accounting complicate attribution
Cpus_allowed_list	permitted logical CPUs	permission is not current placement
context-switch counters	voluntary/involuntary classifications	definitions are kernel-specific; no causal trace
Cap*/Uid/Gid	credentials/capability masks	interpret in namespace/security context

Memory views: maps, RSS, PSS, and working set

- maps: address ranges, permissions, offsets, and backing names.
- RSS: resident pages mapped by a process, including shared pages.
- PSS: shared-page cost divided among mapping processes.
- USS/private pages: pages uniquely attributable under a tool's definition.
- Working set: pages actively needed over a time window; not one static procfs counter.

Attribution problem

Shared libraries, page cache, copy-on-write, deduplication, huge pages, and kernel memory make “memory used by one process” policy-dependent.

Diagnosis

Pair mappings with faults, reclaim, pressure-stall information, and time-series behavior rather than relying on one RSS sample.

Counters versus traces

Counters answer “how much?”

- CPU time, faults, switches, I/O bytes;
- low overhead and aggregatable;
- weak causal ordering.

Traces answer “what happened when?”

- wakeup, block, migration, syscall events;
- supports latency decomposition;
- greater volume and perturbation risk.

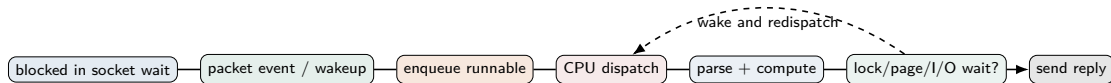
Tool matching

Use `ps/procfs` for snapshots, `pidstat/perf stat` for rates and counters, and `sched-uler/eBPF/perf` traces for wakeup-to-run latency and causality.

Road map

- 1 The Process Abstraction
- 2 Lifecycle, Creation, and Relationships
- 3 Kernel Records and Scheduling State
- 4 Multicore Placement and Resource Control
- 5 Observing Processes on Linux
- 6 Synthesis

End-to-end example: a server thread handles a request



- Kernel task state records lifecycle; shared process objects supply memory, descriptors, and credentials.
- Scheduler queues and wakeup placement determine runnable delay; locks, faults, and devices create blocked intervals.
- A process-level average can hide one pathological worker thread or CPU migration pattern.

Misconceptions corrected

Misconception	⇒	Precise statement
The scheduler runs processes.	⇒	It normally schedules threads/tasks; process is a resource boundary.
Ready means loaded in main memory.	⇒	Runnable means eligible for a CPU; paging/residency is a separate concern.
One PCB contains everything.	⇒	Kernels decompose per-thread and shared state across referenced objects.
Context switch saves every register every time.	⇒	Saved state depends on ABI, architecture, path, and lazy/eager policy.
A zombie still owns its full memory.	⇒	Most resources are gone; exit/accounting identity remains for reaping.
Every orphan is adopted by host PID 1.	⇒	Subreapers and PID namespaces determine the reparenting target.
Background means detached from the terminal.	⇒	It means not in the foreground process group; terminal/session links may remain.
/proc/PID/status is the PCB.	⇒	It is a formatted, partial, racing projection of kernel state.

Key takeaways

- A process is a protected, shared execution environment; threads are the schedulable instruction streams.
- Creation and execution are distinct: `fork()` creates identity/state, while `execve()` replaces the program image.
- Kernel process state is an object graph with independent lifetimes, sharing rules, and synchronization.
- Context switching includes direct kernel work and indirect cache/TLB/NUMA costs.
- Parent trees, process groups, namespaces, and cgroups are separate relationship structures.
- Observation is sampling: correlate per-thread snapshots, counters, and traces before drawing causal conclusions.

Transfer question

When a task blocks, ask: what condition makes it runnable, which queue records the wait, who performs the wakeup, where will it run, and what shared state must remain valid?

Laboratory: follow a process through its lifecycle

1. Write a parent that starts two children, redirects one pipe, and reaps both with `waitpid()`.
2. Observe PID, PPID, PGID, SID, TID, state, CPU, and descriptors using `ps` and `procfs`.
3. Stop/continue one process and record state changes.
4. Deliberately delay reaping to observe a zombie, then reap it.
5. Trace scheduling/wakeup events and explain runnable versus blocked time.
6. Repeat inside a PID namespace/container and compare visible identities.

Safety

Use only processes you own. Do not alter system-wide `/proc/sys` settings for this laboratory.

Next week: CPU scheduling

- objectives: response time, turnaround, fairness, throughput, deadlines;
- FCFS, SJF/SRTF, round robin, priorities, and feedback queues;
- starvation, aging, priority inversion, and bounded blocking;
- multicore load balancing, affinity, topology, and energy;
- quantitative schedule construction and metric calculation.

Preparation

Create a timeline for three jobs with different arrival and service times. Compute response, waiting, and turnaround under FCFS and round robin.

Road map

7 Appendix

Knowledge check: abstraction and lifecycle

1. Can two threads of one process simultaneously be in R and S states?
2. Which process attributes survive `execve()`, and why is close-on-exec important?
3. Why can a large `fork()` be expensive even if the child immediately executes another program?
4. What information remains for a zombie, and what has already been released?
5. Why can a PID-based kill script target the wrong process?
6. Distinguish parent/child, process-group, cgroup, and namespace relationships.

Knowledge check: switching and queues

1. Does every timer interrupt cause a context switch to another thread?
2. When can switching between threads avoid an address-space change?
3. Give three indirect costs not captured by timing low-level switch code.
4. What event moves a blocked task to a run queue?
5. Why are per-CPU run queues scalable, and what new problem do they create?
6. Why is one context-switch counter insufficient to diagnose latency?

Robust child creation and reaping skeleton

```
1 pid_t pid = fork();
2 if (pid < 0) fail("fork");
3 if (pid == 0) {
4     execlp("worker", "worker", (char *)NULL);
5     static const char msg[] = "exec failed\n";
6     (void)write(STDERR_FILENO, msg, sizeof msg - 1);
7     _exit(127);
8 }
9
10 int status;
11 while (waitpid(pid, &status, 0) < 0) {
12     if (errno != EINTR) fail("waitpid");
13 }
14 if (WIFEXITED(status)) report_exit(WEXITSTATUS(status));
15 else if (WIFSIGNALED(status)) report_signal(WTERMSIG(status));
```

A production supervisor must also define descriptor inheritance, signal masks, process groups, timeouts, cancellation, and what to do if the parent crashes.

Exercise: classify state ownership

For each item, classify it as usually per-thread, process-shared, separately reference-counted, or policy-dependent:

1. program counter and user stack;
2. virtual-memory mapping table;
3. file descriptor and open-file description;
4. signal disposition and signal mask;
5. CPU affinity and cgroup membership;
6. credentials and namespace membership;
7. parent relationship and process group;

Extension

For `fork()`, `pthread_create()`, and `execve()`, state whether each item is copied, shared, reset, preserved, or replaced.

Exercise: diagnose a stalled service

Symptoms: low CPU utilization, high request latency, several threads in D, and rising involuntary context-switch counts.

1. List at least four competing hypotheses.
2. Which per-thread procfs fields and counters would you sample?
3. Which trace events establish wakeup-to-run versus device-wait latency?
4. How could CPU affinity or NUMA placement contribute?
5. Why does a high context-switch count not prove context switches are the root cause?

Exercise: compute direct switch overhead

Suppose a workload runs for a full 8 ms quantum and incurs one measured direct switch cost of 6 μ s at each handoff.

1. Estimate the direct overhead fraction:

$$\frac{6 \mu s}{8 \text{ ms} + 6 \mu s}.$$

2. Repeat for a 200 μ s quantum.
3. Explain why neither number estimates cache, TLB, migration, queueing, or lock effects.
4. Give a workload in which a shorter quantum improves user-visible latency despite higher overhead.

Further study

- Linux task model: task structures, shared objects, clone flags, pidfds.
- Scheduling internals: run queues, wakeup placement, scheduler classes.
- Process supervision: subreapers, service managers, process groups, signals.
- Resource isolation: cgroups, namespaces, pressure-stall information.
- Observability: scheduler tracing, eBPF, perf, off-CPU analysis.
- Assurance: task-lifetime races, RCU, reference counting, formal state machines.

Questions?